```
import nltk
import re
import pandas as pd
import math

from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, WordNetLemmatizer


nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt to
[nltk_data]     C:\Users\Prajakta\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\Prajakta\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\Prajakta\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\Prajakta\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\Prajakta\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
```

```
True
```

```
text = """Tokenization is the first step in text analytics.
The process of breaking down a text paragraph into smaller chunks such
as words or sentences is called Tokenization."""

sentences = sent_tokenize(text)
print("Sentences:\n", sentences)
```

```
Sentences:
 ['Tokenization is the first step in text analytics.', 'The process of
breaking down a text paragraph into smaller chunks such as words or
sentences is called Tokenization.']
```

```
words = word_tokenize(text)
print("\nWords:\n", words)
```

```

Words:
```

```
 ['Tokenization', 'is', 'the', 'first', 'step', 'in', 'text',
'analytics', '.', 'The', 'process', 'of', 'breaking', 'down', 'a',
'text', 'paragraph', 'into', 'smaller', 'chunks', 'such', 'as',
'words', 'or', 'sentences', 'is', 'called', 'Tokenization', '.']

stop_words = set(stopwords.words("english"))

text2 = "How to remove stop words with NLTK library in Python?"
text2 = re.sub('[^a-zA-Z]', ' ', text2)

tokens = word_tokenize(text2.lower())
filtered = [w for w in tokens if w not in stop_words]

print("\nTokenized:", tokens)
print("Filtered:", filtered)


Tokenized: ['how', 'to', 'remove', 'stop', 'words', 'with', 'nltk',
'library', 'in', 'python']
Filtered: ['remove', 'stop', 'words', 'nltk', 'library', 'python']

ps = PorterStemmer()
e_words = ["wait", "waiting", "waited", "waits"]

print("\nStemming:")
for w in e_words:
    print(w, "->", ps.stem(w))


Stemming:
wait -> wait
waiting -> wait
waited -> wait
waits -> wait

lemmatizer = WordNetLemmatizer()
text3 = "studies studying cries cry"

print("\nLemmatization:")
for w in word_tokenize(text3):
    print(w, "->", lemmatizer.lemmatize(w))


Lemmatization:
studies -> study
studying -> studying
cries -> cry
cry -> cry

data = "The pink sweater fit her perfectly"
words = word_tokenize(data)
```

```
print("\nPOS Tagging:")
for word in words:
    print(nltk.pos_tag([word]))
```

```
POS Tagging:
[('The', 'DT')]
[('pink', 'NN')]
[('sweater', 'NN')]
[('fit', 'NN')]
[('her', 'PRP$')]
[('perfectly', 'RB')]
```

```
documentA = 'Jupiter is the largest Planet'
documentB = 'Mars is the fourth planet from the Sun'

bagA = documentA.split()
bagB = documentB.split()

uniqueWords = set(bagA).union(set(bagB))

numA = dict.fromkeys(uniqueWords, 0)
for word in bagA:
    numA[word] += 1

numB = dict.fromkeys(uniqueWords, 0)
for word in bagB:
    numB[word] += 1

def computeTF(wordDict, bag):
    tfDict = {}
    total = len(bag)
    for word, count in wordDict.items():
        tfDict[word] = count / total
    return tfDict

tfA = computeTF(numA, bagA)
tfB = computeTF(numB, bagB)

def computeIDF(documents):
    N = len(documents)
    idfDict = dict.fromkeys(documents[0].keys(), 0)

    for doc in documents:
        for word, val in doc.items():
            if val > 0:
                idfDict[word] += 1

    for word, val in idfDict.items():
        idfDict[word] = math.log(N / float(val), 10)

    return idfDict
```

```
idfs = computeIDF([numA, numB])

def computeTFIDF(tf, idfs):
    tfidf = {}
    for word, val in tf.items():
        tfidf[word] = val * idfs[word]
    return tfidf

tfidfA = computeTFIDF(tfA, idfs)
tfidfB = computeTFIDF(tfB, idfs)

df = pd.DataFrame([tfidfA, tfidfB])
print("\nTF-IDF Matrix:\n")
print(df)
```

```
TF-IDF Matrix:

     planet      Mars       Sun      from   largest    Planet
Jupiter   is  \
0  0.000000  0.000000  0.000000  0.000000  0.060206  0.060206
0.060206  0.0
1  0.037629  0.037629  0.037629  0.037629  0.000000  0.000000
0.000000  0.0

   the    fourth
0  0.0  0.000000
1  0.0  0.037629
```